Contents

Networking — Interview Questions & Answers **1**

Networking — Interview Questions & Answers

Q1: What happens when you type `https://google.com` in a browser? > 1. Browser checks cache (DNS cache, HTTP cache) > 2. DNS resolution: OS → /etc/hosts → resolv.conf → recursive resolver → root NS → .com NS → google.com NS → A record > 3. TCP 3-way handshake to port 443 > 4. TLS handshake: ClientHello → ServerHello + Cert → Key exchange → session key → Finished > 5. HTTP/2 GET request with headers (Host, User-Agent, Accept, Cookie, etc.) > 6. Server processes request, response traverses CDN/LB/app servers > 7. Browser renders HTML, parallel requests for CSS/JS/images > 8. Service worker may intercept if PWA

Q2: What is the difference between NAT and PAT? > **NAT (Network Address Translation):** Maps private IP to public IP (1:1 mapping) > **PAT (Port Address Translation)** / NAPT: Many private IPs share one public IP using different source ports (many:1 mapping) — what home routers do

Q3: Explain BGP vs OSPF. > **OSPF:** Interior Gateway Protocol (IGP). Used within an organization. Link-state routing, builds complete topology map, uses Dijkstra algorithm. Fast convergence. > **BGP:** Exterior Gateway Protocol (EGP). Used between Autonomous Systems (internet routing). Path-vector protocol. Policy-based routing. Used by cloud providers and ISPs.

Q4: What is a VLAN and why is it used? > Virtual LAN logically segments a physical network. Benefits: security isolation, broadcast domain reduction, simplified management. Layer 2 concept using 802.1Q tagging. DevOps use: separate management, production, development traffic on same physical switches.

Q5: How does a service mesh handle mTLS? > Each sidecar (Envoy) receives certificates from the control plane's CA (Citadel/istiod). When service A calls service B: A's sidecar → establish TLS with B's sidecar using service account certificates → verify identity. Neither application knows about TLS — it's transparent.

Q6: Explain the difference between ClusterIP, NodePort, and LoadBalancer in Kubernetes. > - **ClusterIP:** Internal-only virtual IP, accessible within cluster. Default. > - **NodePort:** Exposes service on a static port (30000-32767) on each node's IP. Accessible from outside. > - **LoadBalancer:** Creates external cloud LB (AWS ELB, GCP GLB). Routes to NodePort internally. > - **Ingress:** L7 routing using HTTP host/path rules — single LB for many services.

Q7: What is an Anycast IP? > Same IP address is assigned to multiple servers in different locations. Routing sends traffic to the topologically nearest server. Used by CDNs, DNS (8.8.8.8 routes to nearest Google DNS server), DDoS mitigation.

Q8: TCP connection resets vs FIN — difference? > **FIN:** Graceful close — one side signals no more data to send. Both sides close independently (4-way). Connection can be half-closed. > **RST (Reset):** Abrupt close — one party refuses or aborts connection. No waiting. Causes: wrong port, firewall reject, application crash, expired session.

Q9: What is CORS and how do you configure it? > Cross-Origin Resource Sharing — browser security policy preventing JS from one origin making requests to another unless server allows it. Server sends `Access-Control-Allow-Origin` header.

```
add_header 'Access-Control-Allow-Origin' 'https://app.example.com';
add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS';
add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type';
```

Q10: Explain HTTP/2 improvements over HTTP/1.1. > - **Multiplexing:** Multiple requests over single TCP connection (no head-of-line blocking) > - **Header compression (HPACK):** Reduces overhead for repeated headers > - **Server push:** Server sends resources before client requests them > - **Binary protocol:** More efficient parsing (not human-readable like HTTP/1.1) > - **Stream prioritization:** Client can indicate request priority

Q11: How does Kubernetes networking work (CNI)? > Every pod gets unique cluster-wide IP (no NAT). CNI plugin (Flannel, Calico, Cilium) implements: > 1. Creates veth pair for pod → connects to node's network namespace > 2. Sets up routes so pods can reach each other across nodes > 3. Manages overlay network (VXLAN, IPinIP, BGP) > Calico/Cilium also enforce NetworkPolicy using eBPF/iptables.

Q12: What is eBPF and why is it important for networking? > Extended Berkeley Packet Filter — runs sandboxed programs in the Linux kernel without changing kernel code. Used by Cilium for: > - Zero-overhead pod networking (replaces iptables for kube-proxy) > - Network policy enforcement at kernel level > - Observability (Hubble) > - Load balancing > Faster than iptables at scale (iptables is O(n), eBPF is O(1) with hash maps).

Q13: Describe a scenario where you debugged a network issue in production. > *Sample answer structure:* > - Symptom: High latency / connection timeouts between service A and B > - Investigation: `ss` showed high TIME_WAIT count → port exhaustion > - Root cause: Connection pool not reusing connections + high request rate > - Fix: Enabled TCP keepalive + connection pooling + increased ephemeral port range > - Prevention: Added metric for TIME_WAIT count with alerting threshold

Q14: What is MTU and why does it matter for containers/VPNs? > Maximum Transmission Unit — largest packet size for a link (1500 bytes for Ethernet). > VPN/overlay networks add headers → effective MTU shrinks (e.g., VXLAN adds 50 bytes → 1450 MTU). > Misconfigured MTU causes "works for small requests, fails for large" symptoms. > Fix: Set correct MTU on container interfaces, check with `ping -M do -s 1472 host`.

Q15: How do you troubleshoot intermittent packet loss?

```
# Continuous ping with statistics
ping -i 0.2 -c 1000 host | tail -2

# MTR for path analysis
mtr --report --report-cycles 100 host

# Check for errors on interface
ip -s link show eth0        # RX errors, drops

# Check NIC driver issues
ethtool -S eth0 | grep -i error

# Check kernel drops
netstat -s | grep -i "packet receive error"
cat /proc/net/softnet_stat   # softirq drops

# Packet capture
tcpdump -i eth0 -w /tmp/cap.pcap host <destination> &
sleep 30 && kill %1
# Analyze with wireshark
```